Challenge Overview

The challenge provides a remote shell environment via a netcat listener. Upon connecting, it accepts arbitrary user input but processes it through a strict sanitization filter that strips out all whitespace characters and specific environment variables like `$IFS`. This prevents traditional command execution, as standard arguments require spaces.

The objective is to bypass this character restriction to execute system commands and read the contents of `flag.txt`.

Vulnerability Analysis & Bypasses

1. The Space Restriction

When attempting standard commands like `whoami && id && ls -la`, the server outputs the parsed input showing that all spaces have been entirely removed:

Bash
Input: whoami&&id&&ls-la
Error: bash: line 1: whoami: command not found

Because spaces are stripped, the shell interprets the entire string as a single command name rather than a command followed by arguments.

2. Tab/IFS Evasion Failure

A common trick to bypass space filters in Linux is utilizing the Internal Field Separator (`$IFS`) variable or utilizing tabs (`\t`). Testing `$IFS` results in the same failure, indicating either the string `$IFS` is explicitly blacklisted or the filter strips out standard shell variable expansions used for spacing.

3. Brace Expansion Exploitation

In Bash, **Brace Expansion** is a mechanism used to generate arbitrary strings. It follows the syntax `{string1,string2,...}`.

Crucially, when the shell processes a brace expansion containing commas, it separates the comma-delimited elements into distinct arguments *before* executing the binary, completely eliminating the need for spaces.

For example, the input:

Bash
{/bin/ls,-la}

Is expanded by the shell directly into:

Bash
/bin/ls -la

Because this structural syntax relies entirely on brackets and commas, the server's space-stripping filter has no effect on it.

Solution & Exploitation

By leveraging absolute paths (since local path resolution via `$PATH` may fail without environment contexts), we can craft commands utilizing brace expansion to list the directory and read the flag.

Step 1: Enumerate the Directory

Execute an absolute path directory listing to verify the file structure:

Bash
{/bin/ls,-la}

Output:

Plaintext
total 1284
drwxr-xr-x 1 nobody nogroup   4096 May 29 10:43 .
drwxr-xr-x 1 nobody nogroup   4096 May 29 10:43 ..
-rwxr-xr-x 1 nobody nogroup 1298416 May 9 11:07 bash
-rw-r--r-- 1 nobody nogroup   26 May 29 10:38 flag.txt
-rwxr-xr-x 1 nobody nogroup   849 May 29 10:38 run

Step 2: Read the Flag

With the location of `flag.txt` confirmed in the current working directory, use the absolute path of the `cat` binary inside a brace expansion to dump the file contents:

Bash

{/bin/cat,flag.txt}

The shell expands this to `/bin/cat flag.txt`, bypassing the filter entirely and printing the flag.